

Java 8+ Features for Beginners (Tier 2)

A continuation of the Tier 1 guide for Java 8+ features. By now you are comfortable with lambdas, functional interfaces, method references, Optional, default methods, and collection factory methods.

In Tier 2, we cover the bigger features added in Java 14 to 17. These are common in modern Java code and increasingly common in interviews.

What's New in Tier 2?

1. Records (Java 16)
2. Switch expressions (Java 14)
3. The new Date/Time API (java.time)
4. The `var` keyword (Java 10)
5. New String methods (Java 11)
6. Text blocks (Java 15)
7. Sealed classes (Java 17)

Let us go through them one by one.

1. Records (Java 16)

Records are a special kind of class made just for holding data. They save you a lot of boilerplate code.

The problem records solve

Before records, a simple data class looked like this:

```
java
```

```

public class Person {
    private final String name;
    private final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Person)) return false;
        Person p = (Person) o;
        return age == p.age && Objects.equals(name, p.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "'}";
    }
}

```

That is a lot of code for something so simple.

The same thing as a record

```

java

public record Person(String name, int age) {}

```

That one line gives you:

- A constructor that takes `name` and `age`
- Getter methods called `name()` and `age()` (not `getName()` and `getAge()`)
- `equals()` that compares all fields
- `hashCode()` that uses all fields

- `toString()` that shows all fields

Using a record

```
java

Person alice = new Person("Alice", 25);

System.out.println(alice.name());    // Alice
System.out.println(alice.age());     // 25
System.out.println(alice);           // Person[name=Alice, age=25]

Person alice2 = new Person("Alice", 25);
System.out.println(alice.equals(alice2));    // true
```

Notice the getter is `name()`, not `getName()`. This is on purpose. Records use a cleaner naming style.

Adding methods to a record

You can add your own methods.

```
java

public record Person(String name, int age) {
    public boolean isAdult() {
        return age >= 18;
    }

    public String greeting() {
        return "Hello, " + name;
    }
}

Person p = new Person("Alice", 25);
p.isAdult();    // true
p.greeting();   // "Hello, Alice"
```

Validation with compact constructor

If you need to check the input, use a compact constructor.

```
java
```

```

public record Person(String name, int age) {
    public Person {
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name is required");
        }
    }
}

```

Notice there is no parameter list and no `this.x = x`. Java does the assignment for you. You just write the validation.

Static methods and constants

```

java

public record Person(String name, int age) {
    public static final Person ANONYMOUS = new Person("Anonymous", 0);

    public static Person of(String name) {
        return new Person(name, 0);
    }
}

Person p1 = Person.ANONYMOUS;
Person p2 = Person.of("Bob");

```

Records are immutable

You cannot change a record's fields after creation. To get a "changed" version, create a new record.

```

java

Person p = new Person("Alice", 25);
// p.age = 26;    // compile error

Person older = new Person(p.name(), p.age() + 1);

```

This is a feature, not a limitation. Immutable data is easier to reason about and safe to share between threads.

When to use records

- Data transfer objects (DTOs)

- Return values when a method needs to return more than one thing
- Map keys (records have proper `equals` and `hashCode`)
- Working with streams (records are great as intermediate types)

When NOT to use records

- When you need mutable fields
 - When you need to extend another class (records cannot extend anything)
 - When the class has heavy logic, not just data
-

2. Switch Expressions (Java 14)

The old `switch` statement had a few annoying problems:

- You had to write `break` everywhere or risk fall-through bugs
- It was a statement, not an expression, so you could not assign its result
- The syntax was clunky for combining cases

Switch expressions fix all of this.

The old switch

```
java

String day = "MONDAY";
String type;
switch (day) {
    case "MONDAY":
    case "TUESDAY":
    case "WEDNESDAY":
    case "THURSDAY":
    case "FRIDAY":
        type = "Weekday";
        break;
    case "SATURDAY":
    case "SUNDAY":
        type = "Weekend";
        break;
    default:
        type = "Unknown";
}
```

The new switch expression

```
java
String type = switch (day) {
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> "Weekday";
    case "SATURDAY", "SUNDAY" -> "Weekend";
    default -> "Unknown";
};
```

Three big improvements:

1. You can list multiple values in one case
2. The arrow syntax means no fall-through and no `break` needed
3. The whole thing is an expression, so you can assign it to a variable

Notice the semicolon at the end. The whole switch is a single statement.

Multi-line cases with yield

If a case needs more than one line, use a block and `yield` to return a value.

```
java
String type = switch (day) {
    case "MONDAY" -> {
        System.out.println("Start of the week");
        yield "Weekday";
    }
    case "FRIDAY" -> {
        System.out.println("Almost weekend");
        yield "Weekday";
    }
    default -> "Other";
};
```

`yield` is like `return`, but for a switch expression.

Switch on numbers

```
java
```

```
int month = 7;

String season = switch (month) {
    case 12, 1, 2 -> "Winter";
    case 3, 4, 5 -> "Spring";
    case 6, 7, 8 -> "Summer";
    case 9, 10, 11 -> "Autumn";
    default -> "Invalid";
};
```

Switch on enums

When you switch on an enum, the compiler checks that you cover every case. You do not need a default.

```
java

enum Color { RED, GREEN, BLUE }

String name = switch (color) {
    case RED -> "Red";
    case GREEN -> "Green";
    case BLUE -> "Blue";
};
```

If you add a new value to the enum (like `YELLOW`) and forget to handle it here, the compiler tells you. Very helpful.

When to use switch expressions

- When you want to assign a value based on multiple cases
 - When you want safer, cleaner code (no fall-through bugs)
 - When you switch on an enum and want compile-time safety
-

3. Date/Time API (java.time)

Before Java 8, dates and times in Java were a nightmare. The old `Date` and `Calendar` classes were mutable, confusing, and full of traps.

Java 8 introduced `java.time`, based on the popular Joda-Time library. It is clean, immutable, and easy to use.

The main classes

- `LocalDate`: just a date (year, month, day). No time, no timezone.

- `LocalTime`: just a time (hour, minute, second). No date, no timezone.
- `LocalDateTime`: both date and time. No timezone.
- `ZonedDateTime`: date and time with a timezone.
- `Instant`: a moment in time, in UTC.
- `Duration`: a span of time in seconds and nanoseconds.
- `Period`: a span of time in years, months, and days.

LocalDate basics

```
java

LocalDate today = LocalDate.now();
LocalDate birthday = LocalDate.of(1995, 6, 15);
LocalDate parsed = LocalDate.parse("2024-03-15");

System.out.println(today.getYear());           // 2026
System.out.println(today.getMonth());          // APRIL
System.out.println(today.getMonthValue());     // 4
System.out.println(today.getDayOfMonth());     // 26
System.out.println(today.getDayOfWeek());     // SUNDAY
```

Adding and subtracting from a date

```
java

LocalDate today = LocalDate.now();

LocalDate tomorrow = today.plusDays(1);
LocalDate nextWeek = today.plusWeeks(1);
LocalDate nextMonth = today.plusMonths(1);
LocalDate nextYear = today.plusYears(1);

LocalDate yesterday = today.minusDays(1);
LocalDate lastMonth = today.minusMonths(1);
```

These return new `LocalDate` objects. The original is never changed. All `java.time` classes are immutable, which is great.

Comparing dates

```
java
```

```
LocalDate d1 = LocalDate.of(2024, 1, 1);
LocalDate d2 = LocalDate.of(2024, 6, 1);

d1.isBefore(d2);    // true
d1.isAfter(d2);     // false
d1.equals(d2);      // false
d1.isEqual(d2);     // false
```

LocalTime

```
java

LocalTime now = LocalTime.now();
LocalTime meeting = LocalTime.of(14, 30);    // 2:30 PM
LocalTime parsed = LocalTime.parse("14:30:00");

LocalTime later = meeting.plusHours(2);      // 4:30 PM
```

LocalDateTime

For when you need both date and time but do not care about timezone.

```
java

LocalDateTime now = LocalDateTime.now();
LocalDateTime appointment = LocalDateTime.of(2024, 3, 15, 14, 30);

LocalDateTime later = now.plusHours(3).plusDays(1);
```

ZonedDateTime : when timezone matters

```
java

ZonedDateTime here = ZonedDateTime.now();
ZonedDateTime newYork = ZonedDateTime.now(ZoneId.of("America/New_York"));
ZonedDateTime tokyo = ZonedDateTime.now(ZoneId.of("Asia/Tokyo"));
ZonedDateTime delhi = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));
```

Use this when your app deals with users in different time zones.

Period : difference between dates

```
java
```

```
LocalDate start = LocalDate.of(1995, 6, 15);
LocalDate end = LocalDate.of(2026, 4, 26);

Period age = Period.between(start, end);

System.out.println(age.getYears());    // 30
System.out.println(age.getMonths());   // 10
System.out.println(age.getDays());     // 11
```

Duration : difference between times

```
java

LocalTime start = LocalTime.of(9, 0);
LocalTime end = LocalTime.of(17, 30);

Duration workday = Duration.between(start, end);

System.out.println(workday.toHours());    // 8
System.out.println(workday.toMinutes());  // 510

Duration twoHours = Duration.ofHours(2);
Duration thirtySec = Duration.ofSeconds(30);
```

Formatting dates

```
java

LocalDate today = LocalDate.now();

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String formatted = today.format(formatter);
// "26-04-2026"

DateTimeFormatter friendly = DateTimeFormatter.ofPattern("EEE, MMM d, yyyy");
String pretty = today.format(friendly);
// "Sun, Apr 26, 2026"
```

Parsing dates

```
java

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
LocalDate parsed = LocalDate.parse("15-03-2024", formatter);
```

Common patterns

Pattern	Meaning
yyyy	4-digit year
MM	Month number
MMM	Short month name
MMMM	Full month name
dd	Day of month
EEE	Short weekday name
EEEE	Full weekday name
HH	Hour (24h)
hh	Hour (12h)
mm	Minute
ss	Second
a	AM or PM

Quick rule for choosing the right class

- Just a date? Use `LocalDate`.
- Just a time? Use `LocalTime`.
- Both, no timezone? Use `LocalDateTime`.
- Both, with timezone? Use `ZonedDateTime`.
- A point in time for storage or events? Use `Instant`.
- Difference between dates (in days/months/years)? Use `Period`.
- Difference between times (in hours/minutes/seconds)? Use `Duration`.

4. The var Keyword (Java 10)

`var` lets the compiler figure out the type for you when you declare a local variable.

The basics

```
java
```

```
// Old way
String name = "Alice";
int age = 25;
List<String> names = new ArrayList<>();

// With var
var name = "Alice";
var age = 25;
var names = new ArrayList<String>();
```

The variable still has a fixed type. Java just infers it from the right side. This is not dynamic typing like in JavaScript or Python.

Where var really helps

When the type name is long and repetitive.

```
java

// This:
Map<String, List<Person>> peopleByCity = new HashMap<String, List<Person>>>();

// Becomes:
var peopleByCity = new HashMap<String, List<Person>>>();
```

Or in for loops:

```
java

for (var entry : peopleByCity.entrySet()) {
    System.out.println(entry.getKey());
}
```

Where you cannot use var

`var` only works for local variables. You cannot use it for:

- Method parameters
- Method return types
- Class fields
- Variables initialized to `null`
- Lambdas without an explicit type

```
java
```



```

public class Person {
    var name;    // not allowed (field)

    public var getName() {    // not allowed (return type)
        return name;
    }

    public void setName(var n) {    // not allowed (parameter)
        this.name = n;
    }
}

var x = null;    // not allowed, no type to infer
var y = (a) -> a;    // not allowed, lambda needs target type

```

When var helps

- Long generic types
- Obvious right-hand sides like `new ArrayList<String>()`
- For-each loops with clear collection types

When var hurts

When the type is not obvious from looking at the code.

```

java
var x = getValue();    // What is x? You have to look up getValue()

```

In cases like this, write the type explicitly. The goal is readable code, not short code.

Rule of thumb

If a reader can tell the type at a glance, `var` is fine. If not, write the type.

5. New String Methods (Java 11+)

Java 11 added some helpful string methods.

isBlank

Returns `true` if the string is empty or contains only whitespace.

```

java

```

```
"".isBlank();           // true
"  ".isBlank();         // true
"\t\n".isBlank();       // true
"hello".isBlank();      // false
" hello ".isBlank();    // false
```

This is different from `isEmpty()`, which only returns true if the length is zero.

```
java

"  ".isEmpty();  // false
"  ".isBlank();  // true
```

strip, stripLeading, stripTrailing

Remove whitespace, smarter than the old `trim()`.

```
java

" hello ".strip();           // "hello"
" hello ".stripLeading();     // "hello "
" hello ".stripTrailing();   // "  hello"
```

`strip()` handles all Unicode whitespace correctly. `trim()` only handles basic ASCII whitespace, so `strip()` is the modern choice.

repeat

```
java

"ab".repeat(3);    // "ababab"
"-".repeat(20);    // "--------------------"
"hello".repeat(0); // ""
```

Useful for building separators or padding.

lines

Splits a string into a stream of lines.

```
java
```

```
String text = "line1\nline2\nline3";

text.lines().forEach(System.out::println);
// line1
// line2
// line3

long count = text.lines().count();    // 3
```

Other useful methods

```
java

"hello".indent(4);
// "    hello\n"

"Hello, %s!".formatted("Alice");    // Java 15
// "Hello, Alice!"
```

`formatted` is shorthand for `String.format`.

6. Text Blocks (Java 15)

Multi-line strings without all the messy escaping.

The old way

```
java

String json = "{\n" +
    "  \"name\": \"Alice\",\n" +
    "  \"age\": 25\n" +
    "}";
```

Hard to read, hard to write, easy to mess up.

Text blocks

```
String json = """
    {
        "name": "Alice",
        "age": 25
    }
    """;
```

Three double quotes start and end the block. The actual content begins on the next line. No escaping needed for the inner double quotes.

How indentation works

The compiler removes the common leading whitespace from every line.

```
java
String html = """
    <html>
        <body>Hello</body>
    </html>
    """;
```

The result is:

```
<html>
    <body>Hello</body>
</html>
```

Notice the four leading spaces are removed because every line had at least that much indent.

Including special characters

```
java
String quote = """
    She said "hello" and waved.
    """;
```

No need to write `\"`. The content reads naturally.

Backslash for joining lines

If you want to skip a newline:

```
java
```

```
String s = """
    Hello \
    world!
    """;
// "Hello world!\n"
```

The backslash at the end of a line means "no newline here".

Common uses

- JSON literals
- SQL queries
- HTML or XML templates
- Multi-line error messages
- Test data

SQL example

```
java
String sql = """
    SELECT id, name, email
    FROM users
    WHERE age > 18
        AND status = 'active'
    ORDER BY name
    """;
```

Much cleaner than concatenating strings.

7. Sealed Classes (Java 17)

A sealed class controls which other classes can extend it. You explicitly say "only these specific classes can be subclasses."

The problem they solve

Sometimes you want a class hierarchy that is closed. For example, a `Shape` should only be `Circle`, `Square`, or `Triangle`. Nothing else.

Without sealed classes, anyone could write `class Hexagon extends Shape` and break your assumptions.

The basic syntax

```
java

public sealed class Shape permits Circle, Square, Triangle {}

public final class Circle extends Shape {
    double radius;
}

public final class Square extends Shape {
    double side;
}

public final class Triangle extends Shape {
    double base, height;
}
```

The `permits` clause lists the only classes allowed to extend `Shape`. Any other class trying to extend it will not compile.

Why this is useful

The compiler knows the complete list of subclasses. So in a switch expression, it can check that you handle all of them.

```
java

double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.radius * c.radius;
        case Square sq -> sq.side * sq.side;
        case Triangle t -> 0.5 * t.base * t.height;
        // No default needed, all cases covered
    };
}
```

If you later add `Hexagon` to the permits list, the compiler tells you to update this switch. Very helpful.

Three options for subclasses

A class that extends a sealed class must choose one of these three keywords:

1. `final`: cannot be extended further

```
java
```

```
public final class Circle extends Shape {}
```

This is the most common choice.

2. sealed : can only be extended by classes you list

```
java  
  
public sealed class Polygon extends Shape permits Square, Triangle {}
```

Use this for multi-level hierarchies.

3. non-sealed : open to anyone

```
java  
  
public non-sealed class CustomShape extends Shape {}
```

This breaks the seal for that branch. Only use it when you really need to.

Sealed interfaces

The same rules work for interfaces.

```
java  
  
public sealed interface Shape permits Circle, Square, Triangle {}  
  
public record Circle(double radius) implements Shape {}  
public record Square(double side) implements Shape {}  
public record Triangle(double base, double height) implements Shape {}
```

This pairs really well with records, which is a common pattern in modern Java.

When to use sealed classes

- When you have a fixed, known set of types (like shapes, payment methods, response types)
- When you want compile-time exhaustiveness in switch expressions
- When you want to prevent unexpected subclasses
- For domain modeling where the hierarchy is closed

When NOT to use sealed classes

- When the hierarchy is meant to be extended by users
- For frameworks or libraries where flexibility matters
- When you do not know all possible subtypes ahead of time

Putting It All Together

Real examples that mix these features.

Example 1: A simple result type

```
java

public sealed interface Result<T> permits Success, Failure {}

public record Success<T>(T value) implements Result<T> {}
public record Failure<T>(String error) implements Result<T> {}

// Using it
Result<Integer> r = new Success<>(42);

String message = switch (r) {
    case Success<Integer> s -> "Got: " + s.value();
    case Failure<Integer> f -> "Error: " + f.error();
};
```

This is a clean way to handle "success or failure" without exceptions.

Example 2: User profile with validation

```
java
```



```

public record User(String name, String email, LocalDate birthDate) {
    public User {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name required");
        }
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        if (birthDate.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("Birth date in future");
        }
    }

    public int age() {
        return Period.between(birthDate, LocalDate.now()).getYears();
    }

    public boolean isAdult() {
        return age() >= 18;
    }
}

```

Example 3: HTTP response handling

```

java

public sealed interface HttpResponse permits Ok, NotFound, ServerError {}

public record Ok(String body) implements HttpResponse {}
public record NotFound(String path) implements HttpResponse {}
public record ServerError(int code, String message) implements HttpResponse {}

String describe(HttpResponse r) {
    return switch (r) {
        case Ok ok -> "Got data: " + ok.body();
        case NotFound nf -> "Page not found: " + nf.path();
        case ServerError err -> "Server error " + err.code() + ": " + err.message();
    };
}

```

Example 4: Date math for a task scheduler

```

java

```

```

record Task(String name, LocalDateTime dueAt) {
    public boolean isOverdue() {
        return LocalDateTime.now().isAfter(dueAt);
    }

    public Duration timeRemaining() {
        return Duration.between(LocalDateTime.now(), dueAt);
    }
}

var tasks = List.of(
    new Task("Submit report", LocalDateTime.now().plusHours(2)),
    new Task("Review PR", LocalDateTime.now().minusHours(1))
);

tasks.forEach(t -> {
    if (t.isOverdue()) {
        System.out.println(t.name() + " is overdue!");
    } else {
        System.out.println(t.name() + " due in " + t.timeRemaining().toMinutes() + "
});

```

Example 5: Pretty SQL with text blocks

```

java

String userQuery = """
    SELECT u.id, u.name, u.email,
           COUNT(o.id) AS order_count
    FROM users u
    LEFT JOIN orders o ON o.user_id = u.id
    WHERE u.created_at > ?
           AND u.status = 'active'
    GROUP BY u.id, u.name, u.email
    ORDER BY order_count DESC
    LIMIT 10
    """;

```

Common Mistakes

Mistake 1: Using `getX()` style on records

```
java

record Person(String name, int age) {}

Person p = new Person("Alice", 25);
p.getName();    // does not exist
p.name();       // correct
```

Mistake 2: Trying to extend a record

```
java

record Person(String name) {}

class Employee extends Person {}    // does not compile
```

Records cannot be extended. Use composition or interfaces instead.

Mistake 3: Forgetting `yield` in multi-line switch cases

```
java

String s = switch (x) {
    case 1 -> {
        return "one";    // wrong, "return" exits the method
    }
};

String s = switch (x) {
    case 1 -> {
        yield "one";    // correct
    }
};
```

Mistake 4: Mixing old and new switch syntax

```
java

String s = switch (x) {
    case 1 -> "one";
    case 2: "two";    // mix of -> and : not allowed
};
```

Pick one style and stick with it.

Mistake 5: Using `LocalDateTime` when timezone matters

```
java

// Bad: storing a flight time without timezone
LocalDateTime flight = LocalDateTime.of(2024, 6, 15, 9, 30);
```

If the time has any timezone meaning, use `ZonedDateTime` or `Instant` instead.

Mistake 6: Trying to mutate a `java.time` object

```
java

LocalDate today = LocalDate.now();
today.plusDays(1);    // does nothing!

LocalDate tomorrow = today.plusDays(1);    // correct
```

`java.time` classes are immutable. The methods return a new object.

Mistake 7: Using `var` when type is unclear

```
java

var x = service.process(input);    // what is x?
```

If the type is not obvious from the context, write it out.

Mistake 8: Trying to put nulls in records without thinking

```
java

record User(String name, String email) {}

User u = new User(null, null);    // compiles, but probably bad
```

Records do not stop you from using null. Add validation in a compact constructor if you want to disallow it.

Mistake 9: Forgetting that text blocks have a trailing newline by default

```
java
```

```
String s = ""
    hello
    "";
// "hello\n" (note the newline at end)
```

To skip the trailing newline, end with a backslash:

```
java
String s = ""
    hello\
    "";
// "hello"
```

Quick Cheat Sheet

You want to...	Use
Quick data class with equals, hashCode, toString	<code>record</code>
Validate record fields	Compact constructor
Closed class hierarchy	<code>sealed class</code> with <code>permits</code>
Branch on a value and return	Switch expression
Multi-statement switch case	Block with <code>yield</code>
Switch on enum exhaustively	Switch expression on enum (no default needed)
Just a date	<code>LocalDate</code>
Just a time	<code>LocalTime</code>
Date and time, no timezone	<code>LocalDateTime</code>
Date and time with timezone	<code>ZonedDateTime</code>
Span between dates (years/months/days)	<code>Period</code>
Span between times (hours/minutes/seconds)	<code>Duration</code>
Format a date	<code>DateTimeFormatter.ofPattern(...)</code>
Skip writing a long type name	<code>var</code>

You want to...	Use
Check if string has only spaces	<code>s.isBlank()</code>
Strip whitespace better than trim	<code>s.strip()</code>
Repeat a string	<code>s.repeat(n)</code>
Multi-line string	Text block with <code>"""</code>
Format a string	<code>"...%s...".formatted(arg)</code>

Final Tips

1. Records are the single biggest quality-of-life upgrade in modern Java. Use them for any simple data class.
2. Switch expressions are safer than the old switch. Prefer them for new code.
3. Always use `java.time` classes, never the old `Date` or `Calendar`. The old classes are full of traps.
4. Use `var` when the type is obvious. Write the type out when it is not. Readable code beats clever code.
5. Sealed classes plus records plus switch expressions is a powerful combo. It lets you model your domain cleanly and get compile-time safety.
6. Text blocks are not just for JSON. Use them for any multi-line string literal.
7. Remember the rule of thumb: prefer immutable data structures. Records and `java.time` already follow this. Your own code should too when you can.

You now know the most important Java 8 through Java 17 features. Combined with what you learned in Tier 1 and the streams guides, you have a strong grasp of modern Java.

When you are ready, the next tier covers more advanced topics like `CompletableFuture`, the new HTTP client, virtual threads (Java 21), pattern matching for switch, and record patterns.